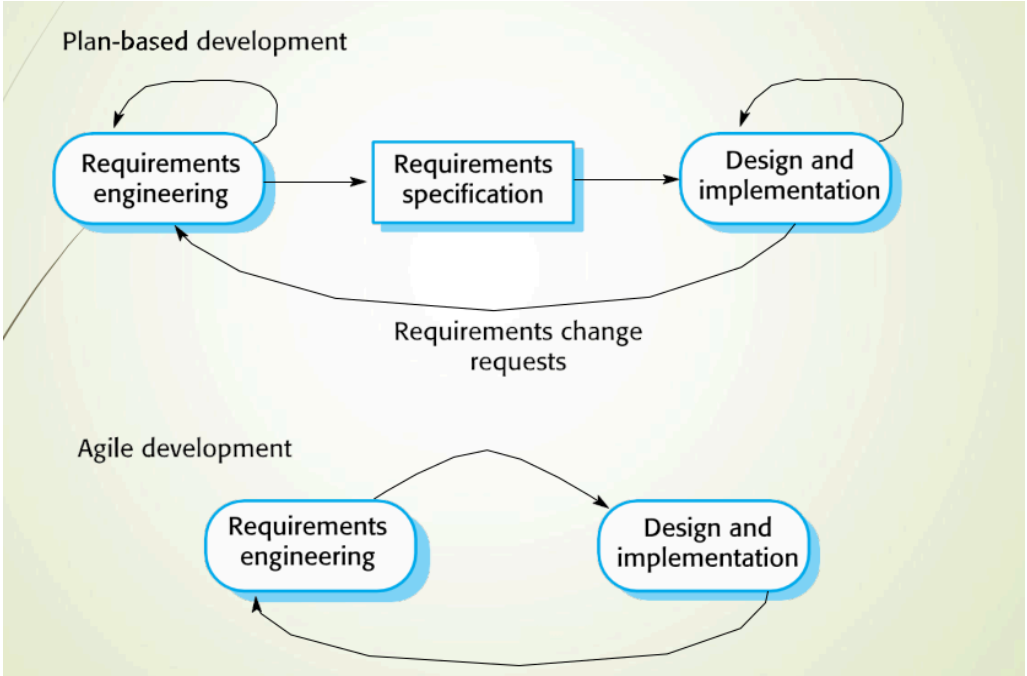
SET Important Questions

Unit 1 Questions

From Agile Software Development (AGILE-LATEST.ppt)

1. Differentiate between Plan-Driven and Agile software development methods with suitable diagrams and examples.

Plan-Driven Development	Agile Development
Development is based on separate planned stages.	Specification, design, implementation, and testing are interleaved.
Requirements are defined and documented in advance.	Requirements can change during development.
Outputs for each stage are planned beforehand.	Outputs are decided through continuous negotiation and feedback.
More emphasis on documentation and planning.	More emphasis on working software and customer collaboration.
Changes are handled through formal change requests.	Changes are welcomed and accommodated quickly.
Suitable for stable requirements and large critical systems.	Suitable for rapidly changing business environments.
Example: Waterfall Model	Example: Scrum, Extreme Programming (XP)



Examples

- **Plan-Driven:** Development of an aircraft control system where requirements are stable and safety is critical.
- **Agile:** Development of an e-commerce or mobile application where customer requirements change frequently.

Conclusion: Plan-driven development focuses on detailed planning and documentation, while Agile development focuses on flexibility, customer collaboration, and rapid delivery of working software.

2. Explain the Agile Manifesto and its twelve principles.

Agile Manifesto

Agile development is based on four core values:

1. **Individuals and interactions over processes and tools**
2. **Working software over comprehensive documentation**
3. **Customer collaboration over contract negotiation**
4. **Responding to change over following a plan**

Agile gives higher priority to the items on the left while recognizing the value of the items on the right.

Twelve Principles of Agile

1. Satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently.
4. Business people and developers must work together daily.
5. Build projects around motivated individuals and trust them.
6. Face-to-face communication is the most effective method of communication.
7. Working software is the primary measure of progress.
8. Promote sustainable development with a constant pace.
9. Continuous attention to technical excellence and good design improves agility.
10. Simplicity is essential; maximize the amount of work not done.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. Teams regularly reflect on how to become more effective and adjust their behavior accordingly.

3. Explain Scrum framework, Scrum roles, artifacts, and Sprint Cycle with a neat diagram.

Scrum Framework

Scrum is an Agile method that focuses on managing iterative development through short development cycles called **Sprints**. It consists of an initial planning phase, a series of sprint cycles, and a project closure phase.

Scrum Roles

Role	Responsibility
Product Owner	Identifies requirements, prioritizes features, and manages the product backlog.
Scrum Master	Ensures Scrum practices are followed and protects the team from external interference.
Development Team	Self-organizing team responsible for developing the software increment.

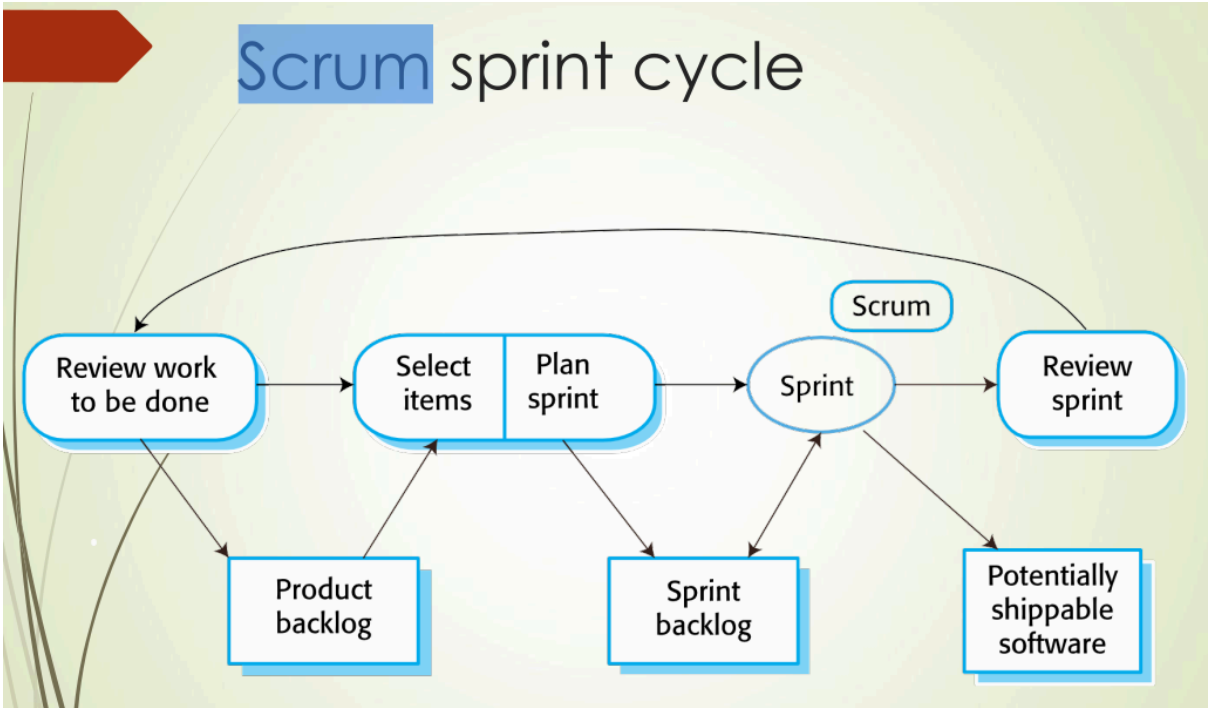
Scrum Artifacts

Artifact	Description
Product Backlog	List of all tasks, requirements, and features to be developed.
Sprint Backlog	Selected backlog items to be completed during a sprint.
Product Increment	Potentially shippable software produced at the end of a sprint.

Sprint Cycle

1. Product backlog is prepared.
2. Features are selected for the sprint.
3. Team develops the selected features during the sprint (2–4 weeks).
4. Daily Scrum meetings are conducted.
5. Sprint review is held at the end of the sprint.
6. Product increment is delivered.
7. Next sprint begins.

Neat Diagram



4. Compare Scrum, Kanban, and Extreme Programming (XP).

Comparison of Scrum, Kanban, and Extreme Programming (XP)

Feature	Scrum	Kanban	Extreme Programming (XP)
Focus	Managing iterative development	Visualizing and improving workflow	Technical excellence and software quality
Work Structure	Fixed-length sprints (2–4 weeks)	Continuous flow of work	Short iterations with frequent releases
Roles	Product Owner, Scrum Master, Development Team	No fixed roles	Customer, Developers, XP Team
Planning	Sprint planning before each sprint	Work items pulled as capacity allows	Planning based on user stories
Meetings	Daily Scrum, Sprint Review, Sprint Planning	No mandatory meetings	Frequent customer interaction

Requirement Changes	Usually added in next sprint	Can be added anytime	Welcomes changes even during development
Main Practices	Backlog management and sprint cycles	Kanban board, WIP limits	Pair programming, TDD, refactoring, continuous integration
Best Suited For	Projects needing structured Agile management	Teams requiring continuous delivery	Projects emphasizing code quality and customer involvement

5. Explain Pair Programming and its benefits in Agile development.

Pair Programming

Pair Programming is an Agile technique used in Extreme Programming (XP) where **two programmers work together at the same computer to develop code**. Pairs are formed dynamically so that team members work with different partners during development.

Benefits of Pair Programming

1. Develops **common ownership** of code.
2. Spreads **knowledge and expertise** across the team.
3. Acts as an **informal code review** since two people examine the code.
4. Encourages **refactoring** and code improvement.
5. Reduces project risk when team members leave.
6. Improves code quality by detecting errors early.
7. Enhances collaboration and communication among developers.

6. Explain the Rapid Application Development (RAD) model and discuss its advantages and disadvantages.

Rapid Application Development (RAD) Model

Rapid Application Development (RAD) is an **incremental software development model** with a short development cycle. It is considered a **high-speed version of the Waterfall model** and enables a development team to build a fully functional system in a very short time period.

Phases of RAD

1. **Requirements Planning** – Define project goals and scope.
2. **User Design** – Build and refine prototypes with user feedback.
3. **Construction** – Rapid development using reusable components and automated tools.
4. **Cutover** – Testing, deployment, and delivery of the final system.

Advantages

- Fast product development.
- Efficient documentation.
- High user interaction and feedback.
- Reduces development time.

Disadvantages

- Users may not be comfortable with rapid development activities.
- Not suitable for projects involving high technical risks.

From Software Requirements (SOFTWARE_REQUIREMENTS.ppt)

7. Explain the concept of Functional Requirements and Non-Functional Requirements with examples.

Functional Requirements

Functional requirements describe the **services or functions** that a system should provide. They specify how the system should react to inputs and behave in different situations.

Examples:

- A user shall be able to search appointment lists for all clinics.
- The system shall generate a daily list of patients with appointments.
- Each staff member shall be identified by an 8-digit employee number.

Non-Functional Requirements

Non-functional requirements define the **quality attributes, constraints, and system properties** such as reliability, response time, storage requirements, standards, and development constraints.

Examples:

- The system should respond within 2 seconds.

- The system should be available 99.9% of the time.
- The software must be developed using Java.
- The system should support 1000 concurrent users.

8. Discuss the Requirement Elicitation and Analysis process. Explain the challenges involved in requirement gathering.

Requirement Elicitation and Analysis

Requirement elicitation and analysis is the process in which software engineers work with stakeholders to understand the application domain, required services, performance needs, constraints, and other system requirements.

Process Stages

1. **Requirements Discovery and Understanding**
    - Gather requirements from stakeholders.
    - Understand business needs and system objectives.
2. **Requirements Classification and Organization**
    - Group related requirements into categories.
    - Organize requirements systematically.
3. **Requirements Prioritization and Negotiation**
    - Identify important requirements.
    - Resolve conflicts among stakeholder requirements.
4. **Requirements Specification**
    - Document the agreed requirements clearly and precisely.

Challenges in Requirement Gathering

1. Stakeholders may not clearly understand their needs.
2. Different stakeholders may have conflicting requirements.
3. Requirements may change during development.
4. Communication gaps between users and developers.
5. Ambiguous or incomplete requirements.
6. Difficulty in prioritizing requirements.
7. Large number of stakeholders can make negotiation difficult.

9. Explain ethnography and prototyping techniques used in requirement engineering. Compare their advantages and limitations.

Ethnography

Ethnography is a requirement engineering technique in which a social scientist spends considerable time observing and analyzing how people actually work. It helps identify social and organizational factors that affect system requirements.

Advantages

- Helps understand actual work practices.
- Identifies social and organizational factors.
- Reveals requirements that users may not explicitly state.
- Effective for understanding existing processes.

Limitations

- Time-consuming.
- Cannot identify new features that should be added to the system.
- Studies existing practices that may no longer be relevant.

Prototyping

Prototyping is a technique where a prototype of the system is built, tested, and refined repeatedly until an acceptable version is achieved.

Advantages

- Provides clarity of requirements.
- Helps identify risks early.
- Encourages user involvement.
- Takes less time to complete.

Limitations

- High cost.
- Slow process due to repeated modifications.
- Too many changes may occur.

Comparison of Ethnography and Prototyping

Ethnography	Prototyping
Based on observation of users at work.	Based on building a working model of the system.
Focuses on understanding existing processes.	Focuses on validating and refining requirements.
Identifies hidden user needs.	Obtains direct user feedback.
Does not help identify new features easily.	Helps discover and improve features.
Time-consuming.	May increase cost due to repeated revisions.

10. Describe the various notations used in Software Requirement Specification (SRS) and their significance.

Notations Used in Software Requirement Specification (SRS)

1. **Natural Language**
    - Requirements are written as numbered sentences in simple language.
    - Easy for customers and users to understand.
2. **Structured Natural Language**
    - Requirements are written using a standard format or template.
    - Provides consistency and reduces ambiguity.
3. **Design Description Languages**
    - Uses a language similar to a programming language with abstract features.
    - Useful for interface specifications and operational models.
4. **Graphical Notations**
    - Uses diagrams and models such as UML use case diagrams and sequence diagrams.
    - Helps visualize functional requirements and system behavior.
5. **Mathematical Specifications**
    - Uses mathematical concepts such as finite-state machines and sets.
    - Provides precise and unambiguous specifications.

Significance

- Improves clarity and understanding of requirements.
- Reduces ambiguity and misunderstandings.
- Helps in communication between stakeholders and developers.
- Provides a basis for system design, implementation, and testing.
- Ensures requirements are documented in a consistent and organized manner.

11. Explain requirement validation and requirement prioritization techniques.

Requirement Validation

Requirement validation is the process of checking whether the requirements define the system that the customer really wants. It ensures that requirements are correct, complete, consistent, and testable.

Techniques of Requirement Validation

1. **Requirements Reviews**
    - Regular reviews are conducted while requirements are being defined.
    - Both customers and developers participate.
    - Reviews may be formal or informal.

2. **Review Checks**
    - **Verifiability** – Can the requirement be tested?
    - **Comprehensibility** – Is the requirement clearly understood?
    - **Traceability** – Is the source of the requirement known?
    - **Adaptability** – Can the requirement be changed easily?

Requirement Prioritization Techniques

Requirement prioritization is the process of ranking requirements according to their importance and business value so that the most critical requirements are implemented first.

Techniques

1. **High, Medium, Low Priority**
    - Requirements are classified based on importance.
2. **MoSCoW Technique**
    - Must Have
    - Should Have
    - Could Have
    - Won't Have
3. **Stakeholder-Based Prioritization**
    - Requirements are prioritized based on stakeholder needs and business goals.
4. **Negotiation**
    - Conflicting requirements are discussed and resolved among stakeholders.

12. Describe the stakeholders involved in a Mental HealthCare Patient Management System.

Stakeholders in a Mental HealthCare Patient Management System (MHC-PMS)

1. **Patients**
    - Their personal and medical information is recorded in the system.
2. **Doctors**
    - Assess and treat patients.
3. **Nurses**
    - Coordinate consultations with doctors and administer treatments.
4. **Medical Receptionists**
    - Manage patient appointments.
5. **IT Staff**
    - Install, maintain, and support the system.
6. **Medical Ethics Manager**

- Ensures that the system complies with ethical guidelines for patient care.

7. **Health Care Managers**
   - Obtain management information and reports from the system.
8. **Medical Records Staff**
   - Ensure that patient records are maintained, preserved, and managed properly.

13. Discuss the importance of use case diagrams in system modeling.

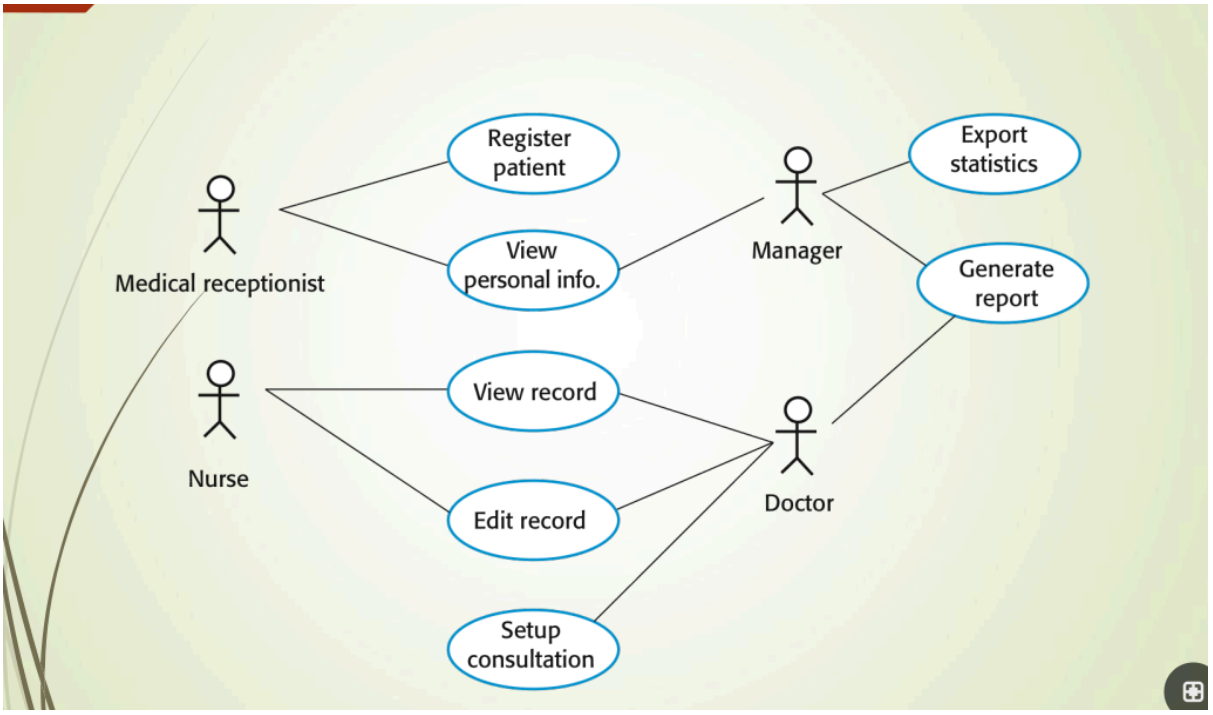
Importance of Use Case Diagrams in System Modeling

A Use Case Diagram is a graphical representation that shows the interactions between **actors** (users or external systems) and the **system**. It helps in modeling the functional requirements of a system.

Importance

1. Helps understand system functionality from the user's perspective.
2. Identifies different actors interacting with the system.
3. Clearly shows the services provided by the system.
4. Improves communication between users, customers, and developers.
5. Helps in requirement analysis and validation.
6. Provides a simple graphical view of system requirements.
7. Serves as a basis for system design and testing.

Diagram



Unit 2 Questions

From System Modelling (unit-2_chapter1-UPDATED.pptx)

1. Explain context models and their applications with examples.

Context Models

Context models are used to show the **operational context of a system**. They illustrate the boundaries of the system and its interactions with external systems, organizations, or users. Context models help identify the environment in which the system operates.

Applications of Context Models

1. Define the system boundary.
2. Identify external systems interacting with the system.
3. Help understand system dependencies.
4. Assist in requirements elicitation and analysis.
5. Improve communication among stakeholders.
6. Provide a high-level view of the system environment.

Example

In a **Mental HealthCare Patient Management System (MHC-PMS)**, the system may interact with:

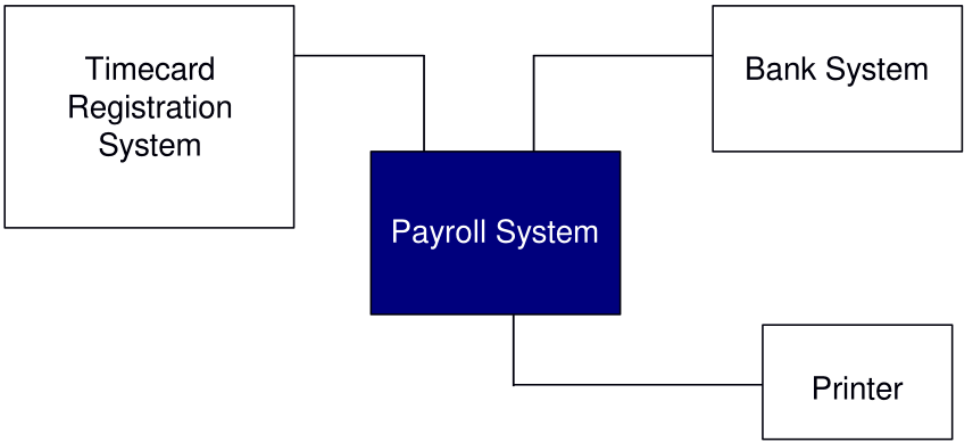
- Patient record databases
- Appointment systems
- Hospital administration systems
- Medical staff and patients

These external entities and their interactions with the MHC-PMS are represented using a context model.

Diagram

Example: The context of Payroll System

High-level architectural models show the system and its relationship with other systems.



High-level architectural model; expressed as a block diagram.
Relationships are not defined => need to be supplemented by other models.

2. Discuss the importance of use case diagrams in system modeling.

Importance of Use Case Diagrams in System Modeling

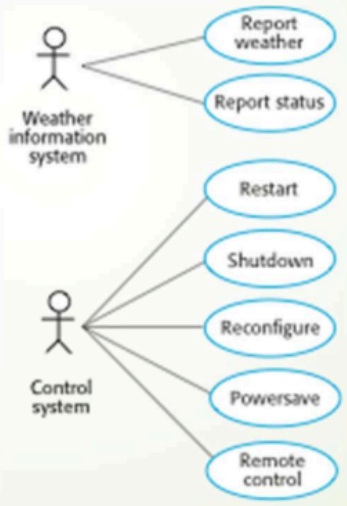
A Use Case Diagram is a graphical model that shows the interactions between **actors** and the **system**. It is used to represent the functional requirements of a system.

Importance

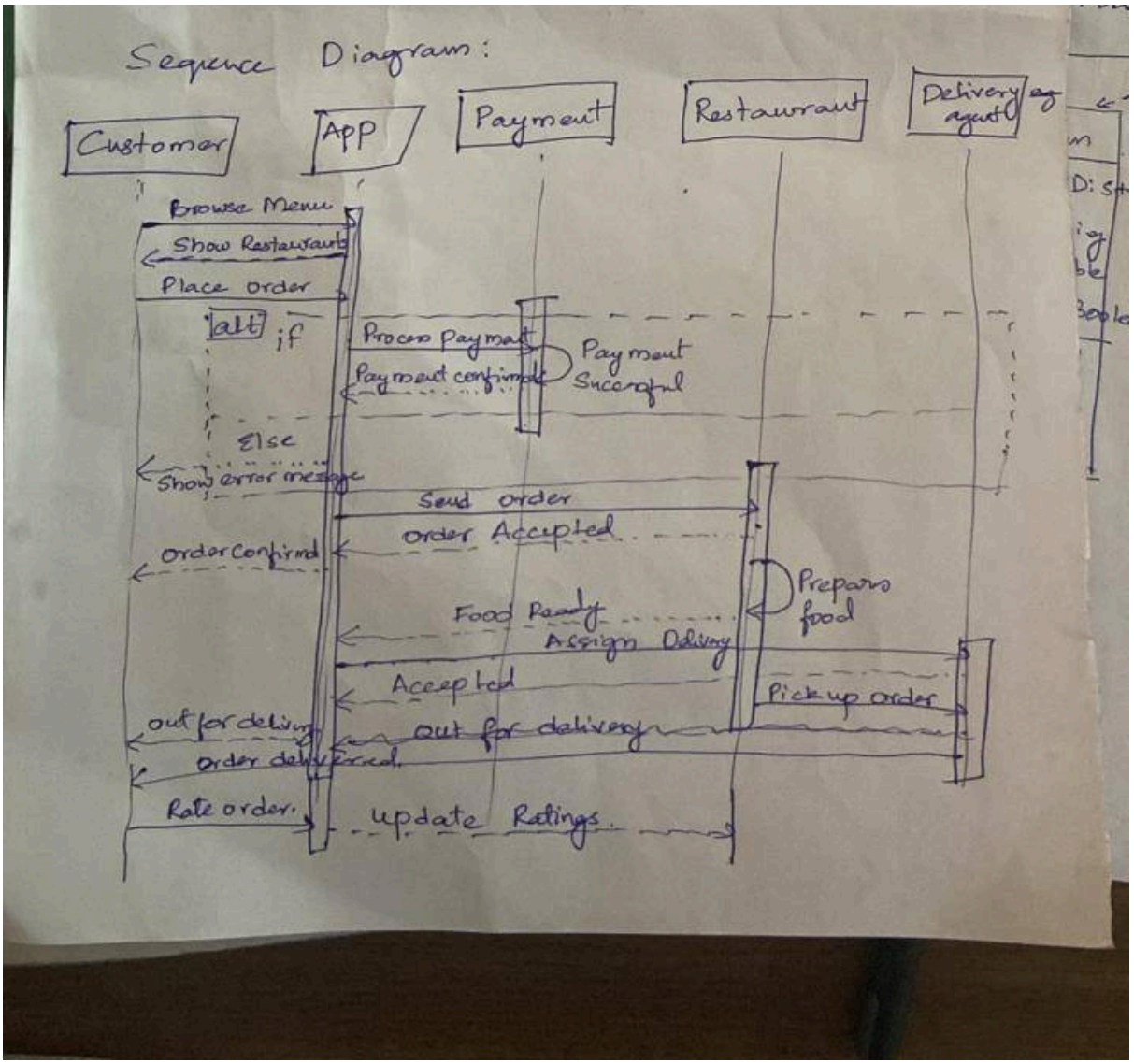
1. Provides a clear view of system functionality.
2. Identifies the actors interacting with the system.
3. Shows how users interact with different system services.
4. Helps in understanding and validating requirements.
5. Improves communication between stakeholders and developers.
6. Supports system analysis and design activities.
7. Acts as a basis for test case development.

Diagram

Weather station use cases



3. Construct a sequence diagram for an Online Food Delivery System and explain the message flow.



4. Explain UML State Machine Modeling with a suitable example such as a microwave oven or ATM system.

UML State Machine Modeling

UML State Machine Modeling is used to describe the dynamic behavior of a system by showing the different states of an object and the events that cause transitions between those states.

ATM System Example

The ATM system can be represented using the following states:

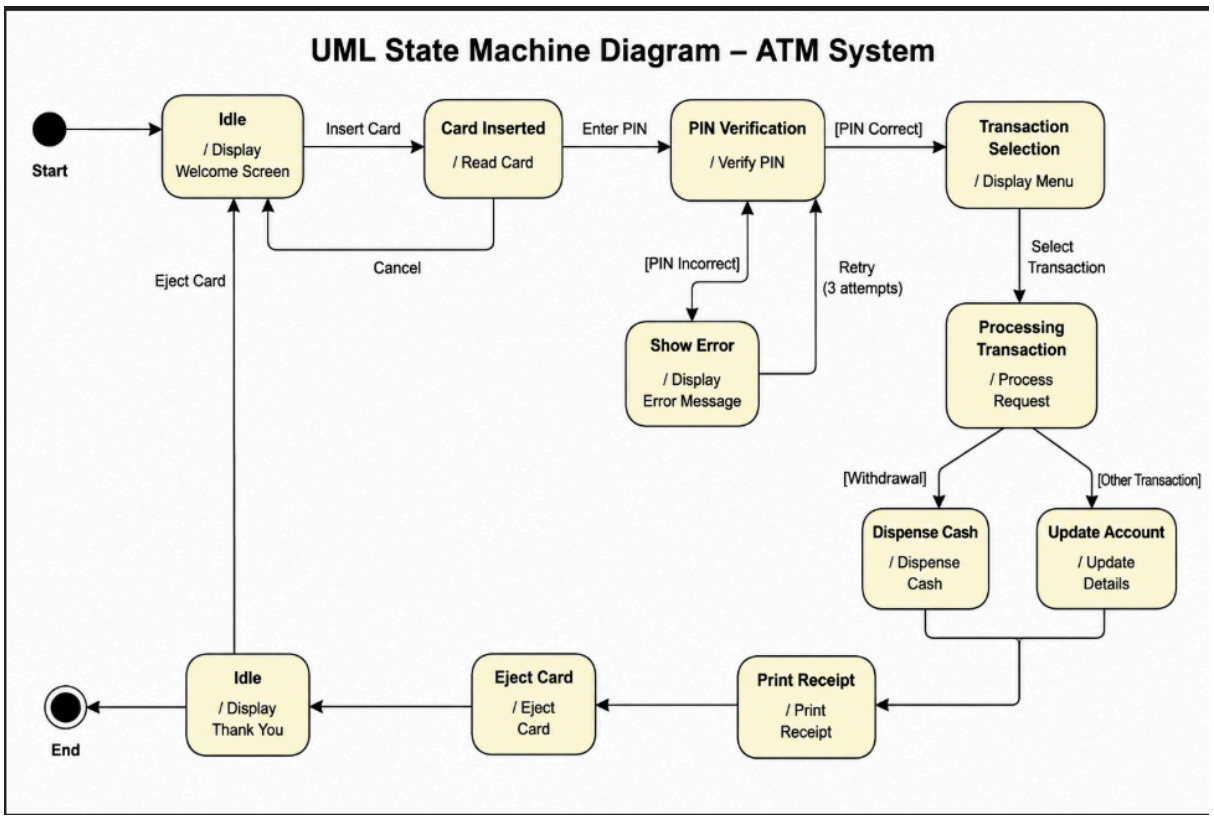
1. **Idle State** – ATM waits for a card.

2. **Card Inserted** – User inserts the ATM card.
3. **PIN Verification** – System verifies the entered PIN.
4. **Transaction Selection** – User selects a transaction.
5. **Processing Transaction** – ATM processes the request.
6. **Dispense Cash** – Cash is dispensed if withdrawal is selected.
7. **Print Receipt** – Receipt is generated.
8. **Eject Card** – ATM returns the card.
9. **Idle State** – System returns to the initial state.

Advantages

- Represents dynamic behavior clearly.
- Shows state transitions caused by events.
- Helps in analyzing and designing reactive systems.
- Useful for systems whose behavior depends on their current state.

Diagram



5. Explain object aggregation and composition in UML with suitable examples.

Object Aggregation

Aggregation represents a **whole-part relationship** where the part objects can exist independently of the whole object. It is a weak form of association.

Example:

- Department — Employee
- A department contains employees, but employees can exist even if the department is removed.

Object Composition

Composition represents a **strong whole-part relationship** where the part objects cannot exist independently of the whole object.

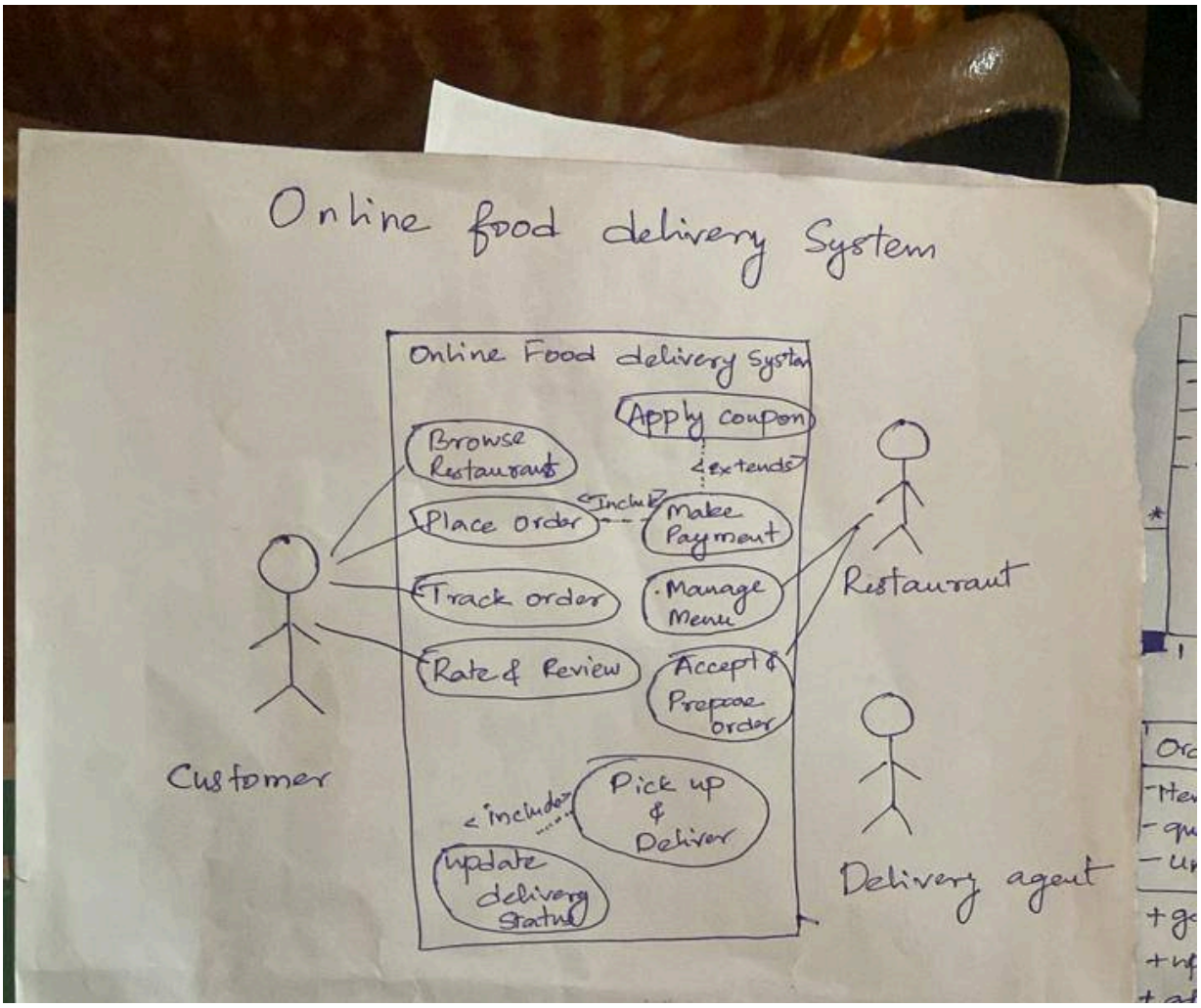
Example:

- House — Room
- A room cannot meaningfully exist without the house to which it belongs.

Difference Between Aggregation and Composition

Aggregation	Composition
Weak relationship	Strong relationship
Parts can exist independently	Parts cannot exist independently
Lifetime of part is independent of whole	Lifetime of part depends on whole
Represented by a hollow diamond	Represented by a filled diamond
Example: Department–Employee	Example: House–Room

6. Draw and explain a Use Case Diagram for an Online Shopping System.



From Design and Implementation (UNIT-2_CHAPTER_2-SOFTWARE_DESIGN.pptx)

7. Discuss the different software design models and their role in software architecture.

Software Design Models

Software design models are used to represent different views of a system during the design process. They help developers understand the system structure, behavior, and interactions.

Types of Software Design Models

1. **Subsystem Models**
    - Show logical grouping of objects into subsystems.
    - Help organize the overall system architecture.
2. **Sequence Models**
    - Show the sequence of interactions between objects.
    - Describe how objects communicate to perform a task.
3. **State Machine Models**
    - Show how objects change state in response to events.
    - Useful for modeling dynamic behavior.
4. **Use Case Models**
    - Show interactions between actors and the system.
    - Help capture functional requirements.
5. **Aggregation Models**
    - Show whole-part relationships between objects.
6. **Generalization Models**
    - Show inheritance relationships among classes.

Role in Software Architecture

1. Provide different views of the system.
2. Help in understanding system structure and behavior.
3. Support communication among developers and stakeholders.
4. Assist in system analysis and design.
5. Improve documentation and maintainability.
6. Serve as a blueprint for implementation.

8. Describe object class identification with reference to a weather station system.

Object Class Identification

Object class identification is the process of identifying the classes or objects required in a system and defining their responsibilities and relationships.

Weather Station System

In a weather station system, the following object classes can be identified:

1. **WeatherStation**
    - Collects and manages weather data.
2. **WeatherData**
    - Stores temperature, pressure, humidity, and wind information.

3. **GroundThermometer**
    - Measures ground temperature.
4. **Anemometer**
    - Measures wind speed and direction.
5. **Barometer**
    - Measures atmospheric pressure.
6. **RainGauge**
    - Measures rainfall.
7. **WeatherInformationSystem**
    - Receives and processes weather data from the weather station.

Importance

1. Helps in organizing system functionality into classes.
2. Improves modularity and maintainability.
3. Forms the basis for object-oriented design.
4. Defines relationships between system components.

9. Explain the role of Continuous Integration and Continuous Deployment in DevOps. *(partially — CI/CD is in design context)*

Continuous Integration (CI)

Continuous Integration is a development practice where developers frequently integrate their code into a shared repository. Each integration is automatically built and tested to detect errors early.

Benefits

1. Detects integration errors early.
2. Improves software quality.
3. Reduces development time.
4. Supports frequent code updates.

Continuous Deployment (CD)

Continuous Deployment is a practice where software changes that pass automated testing are automatically deployed to production.

Benefits

1. Faster software delivery.
2. Reduces manual deployment effort.
3. Ensures quick availability of new features.
4. Improves feedback from users.

Role in DevOps

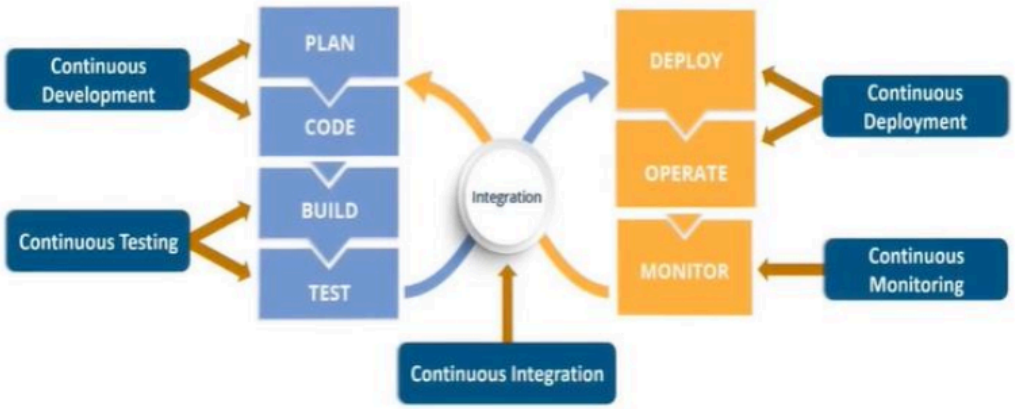
1. Automates software build, testing, and deployment.
2. Enables rapid and reliable software releases.
3. Improves collaboration between development and operations teams.
4. Reduces deployment risks and human errors.
5. Supports continuous delivery of high-quality software.

Unit 3 Questions

From DevOps (Slides 24–40)

1. Illustrate the DevOps lifecycle and explain each stage in detail.

DevOps Lifecycle (Contd.)



DevOps Lifecycle

Stage 1: Continuous Development (Contd.)

- This stage focuses on **planning and coding** the software application.
- Development is carried out in **small, continuous coding cycles**.
- Code can be written in **any programming language**.
- All code is maintained using **Version Control Systems (VCS)**.
- A **central repository** acts as a **single source of truth** for the project.
- Developers collaborate using the **latest committed code**.
- Operations teams can access the same code while **planning releases**.

•Version control enables:

- **•Tracking code changes**
- **•Collaboration among team members**
- **•Rollback to previous stable versions** in case of bugs or failures

•Commonly used tools:

- **•Git**
- **•SVN**
- **•Mercurial**
- **•CVS**
- **•JIRA** (for issue tracking and coordination)

Stage 2: Continuous Integration

•Code from **multiple developers** is frequently merged into a **central repository**.

•New features are **integrated with existing code** early and often.

•Frequent integration helps **detect conflicts and defects early**.

•Continuous Integration reduces **integration problems and build costs**.

•Code is **automatically built, packaged, and tested** after integration.

•Integrated code is sent to **testing or production servers**.

•CI relies on **automated test execution** to reduce manual effort.

•Commonly used CI tools include:

- **•Jenkins**
- **•GitLab CI**

Stage 3: Continuous Testing

•Code is **automatically tested** before deployment to detect bugs and performance issues.

•Testing is performed in **test environments**, often using **Docker containers**.

•**Automation testing** replaces repetitive manual testing and improves accuracy.

•Automated tests are triggered using **Continuous Integration tools**.

•Reduces **bugs** there by improving **reliability**

•**Test reports** are generated automatically, making it easier to identify failures.

Stage 4: Continuous Deployment

Tested code is automatically **deployed** to servers.

Deployment is handled using:

- **Configuration Management tools**
- **Containerization tools**

Ensures **consistent environments** across development, testing, and production.

Helps achieve **consistent and high-quality releases**.

Reduces manual deployment effort and minimizes human errors.

Containerization **packages applications with all dependencies**, ensuring uniform behavior across environments.

Docker is the most widely used containerization tool.

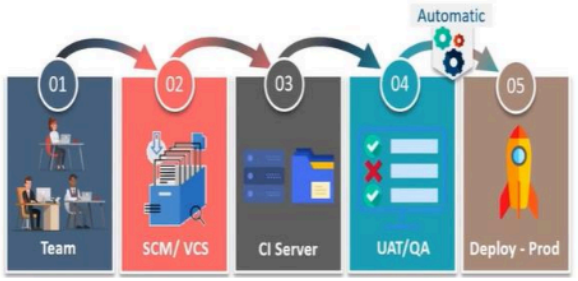
Stage 4: Continuous Deployment (Contd.)

Configuration Management Tools

- Puppet
- Chef
- Ansible
- SaltStack

Cloud Platforms Supporting Containers

- AWS ECS
- Azure Container Service
- Google Container Engine



Stage 5: Continuous Feedback

- System collects **user feedback and performance data**
- Helps understand **real-world usage**
- Feedback is used to plan improvements

Stage 6: Continuous Monitoring

- Application performance is **continuously monitored** after deployment.
- Helps detect:
  - Memory issues
  - Server failures
  - Crashes
  - Unusual system behavior
- Monitoring is critical because some bugs may **escape testing**.
- Monitoring tools provide **early warnings** before users face issues.
- Operations team plays a **major role** in this stage.

- Common monitoring tools:
  - **Splunk-**Splunk **Splunk** is a **data monitoring and log analysis tool** used to collect, search, analyze, and visualize machine-generated data from servers, applications, and networks.
    Eg-A company collects logs from its servers. Splunk analyzes the logs and shows if any **errors, failures, or unusual activities** occur.
  - **Nagios**:To monitor server, network and application
    If server goes down, nagios sends notification or alert to admin
  - **New Relic**-Cloud based monitoring tool

Stage 6: Continuous Monitoring tool (Contd.)

- **ELK Stack (Elasticsearch, Logstash, Kibana)- -** helps in **Log collection, Search and analysis ,Visualization and management**

a) Elasticsearch

- Stores and searches large volumes of data quickly.

b) Logstash

- Collects and processes logs from different sources.

c) Kibana

- Displays data through graphs, charts, and dashboards

Stage 7: Continuous Operations

- Release and update processes are **fully automated**
- Keeps cycles short
- Allows developers to focus on **new features**

- **Faster and more efficient** software development
- **Fewer bugs** and improved user experience
- Better **collaboration** between Dev and Ops teams
- **Higher return on investment (ROI)**
- Automatic deployment supports **large-scale systems**
- Suitable for **both small and large teams**
- Early bug detection through **automated testing & monitoring**
- **Streamlined workflows** using version control and automation
- Continuous feedback improves **product quality**
- Automation reduces effort in testing and deployment

Tools Commonly Used in DevOps

- **Version Control:** Git, GitHub, GitLab
- **CI/CD Tools:** Jenkins, GitHub Actions, GitLab CI
- **Build Tools:** Maven, Gradle
- **Configuration Management:** Ansible, Chef, Puppet
- **Containerization:** Docker
- **Container Orchestration:** Kubernetes
- **Monitoring & Logging:** Prometheus, Grafana, ELK Stack
- **Cloud Platforms:** AWS, Azure, Google Cloud
- **Testing Tools:** Selenium, JUnit, TestNG

2. Compare DevOps and traditional software development approaches.

Comparison Between DevOps and Traditional Software Development

DevOps	Traditional Software Development
Development and operations teams work together.	Development and operations teams work separately.
Continuous integration and continuous deployment are used.	Deployment is usually manual and infrequent.
Frequent software releases.	Releases occur after long development cycles.
High level of automation.	Limited automation.
Continuous testing throughout development.	Testing is usually performed after development.
Faster feedback and issue resolution.	Feedback is received late in the process.
Faster delivery of software updates.	Slower software delivery.
Focuses on collaboration and continuous improvement.	Focuses on completing phases sequentially.
Suitable for dynamic and rapidly changing requirements.	Suitable for stable requirements.
Lower deployment risk due to frequent small changes.	Higher deployment risk due to large releases.

3. Explain the role of Continuous Integration and Continuous Deployment in DevOps.

Continuous Integration (CI)

Continuous Integration is a software development practice in which developers frequently merge their code changes into a shared repository. Each integration is automatically built and tested to detect errors early.

Benefits

1. Detects defects early.
2. Improves software quality.
3. Reduces integration problems.
4. Supports rapid development.

Continuous Deployment (CD)

Continuous Deployment is a practice where software changes that pass all automated tests are automatically deployed to production.

Benefits

1. Faster software delivery.
2. Reduces manual deployment effort.
3. Provides quick feedback from users.
4. Ensures continuous availability of new features.

Role in DevOps

1. Automates build, testing, and deployment processes.
2. Enables frequent and reliable software releases.
3. Improves collaboration between development and operations teams.
4. Reduces deployment errors and risks.
5. Supports continuous delivery of high-quality software.

4. Discuss popular DevOps tools used for version control, build automation, and deployment.

Popular DevOps Tools

DevOps uses various tools to automate software development, testing, integration, and deployment activities.

Version Control Tools

1. **Git**
    - Distributed version control system.
    - Tracks code changes and supports collaboration.
2. **GitHub**
    - Web-based platform for hosting Git repositories.
    - Supports version control and team collaboration.
3. **GitLab**
    - Provides source code management and CI/CD features.

Build Automation Tools

1. **Maven**
    - Build automation and project management tool for Java projects.
    - Manages dependencies and builds applications.
2. **Gradle**
    - Automates building, testing, and deployment.
    - Supports multiple programming languages.
3. **Apache Ant**
    - Java-based build automation tool.

Deployment Tools

1. **Jenkins**
    - Open-source automation server.
    - Supports Continuous Integration and Continuous Deployment (CI/CD).
2. **Docker**
    - Containerization platform used to package and deploy applications consistently.
3. **Kubernetes**
    - Container orchestration tool.
    - Automates deployment, scaling, and management of containers.
4. **Ansible**
    - Configuration management and deployment automation tool.
5. **Terraform**
    - Infrastructure as Code (IaC) tool used to provision and manage infrastructure.

5. Discuss Continuous Monitoring and Continuous Feedback in the DevOps lifecycle.

Continuous Monitoring

Continuous Monitoring is the process of continuously observing applications, infrastructure, and services to ensure they are functioning correctly and meeting performance requirements.

Benefits

1. Detects issues and failures quickly.
2. Improves system reliability and availability.
3. Tracks performance and resource usage.
4. Helps maintain system security.

Continuous Feedback

Continuous Feedback is the process of collecting information from users, monitoring tools, and stakeholders to improve the software continuously.

Benefits

1. Helps identify user needs and expectations.
2. Improves software quality.
3. Supports continuous improvement.
4. Enables faster issue resolution.

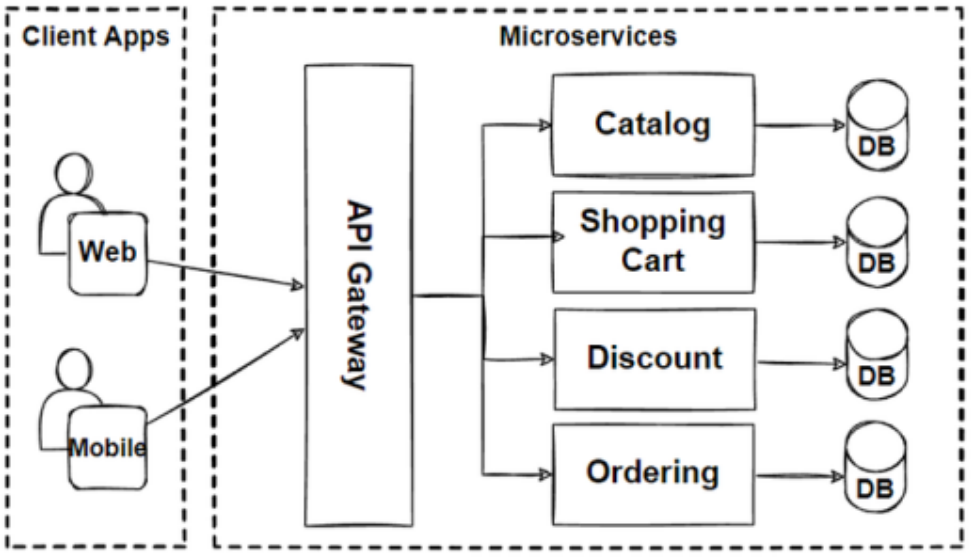
Role in DevOps Lifecycle

1. Provides real-time information about system performance.
2. Helps detect and resolve problems early.
3. Supports informed decision-making.
4. Improves customer satisfaction through regular feedback.
5. Enables continuous enhancement of software and services.

From Microservices Architecture (Slides 64–71)

6. Describe the architecture of a Microservices-based application with a neat block diagram.

A Microservices Architecture is an architectural style in which an application is developed as a collection of small, independent services. Each service focuses on a specific business function and communicates with other services using lightweight mechanisms such as REST or gRPC.



Microservices Architecture

Components

1. **Client Applications**
   - Web, mobile, or desktop applications that access the system.
2. **API Gateway**
   - Single entry point for client requests.
   - Routes requests to appropriate microservices.
3. **Microservices**
   - Independent services such as User Service, Order Service, Payment Service, and Inventory Service.
   - Each service handles a specific business capability.
4. **Database per Service**
   - Each microservice manages its own database.
   - Ensures data ownership and independence.
5. **Communication Layer**
   - Services communicate through APIs, REST, messaging systems, or gRPC.

Advantages

1. Independent deployment of services.
2. Loose coupling between components.
3. Better scalability.
4. Easier maintenance and testing.
5. Improved fault isolation.

7. Compare Monolithic Architecture and Microservices Architecture.

Monolithic Architecture	Microservices Architecture
Entire application is developed as a single unit.	Application is divided into small independent services.
All modules are tightly coupled.	Services are loosely coupled.
Single codebase for the entire application.	Separate codebase for each service.
Entire application is deployed together.	Each service can be deployed independently.
Scaling requires scaling the whole application.	Individual services can be scaled independently.

Failure in one module may affect the entire application.	Failure in one service is isolated from others.
Easier to develop for small applications.	Better suited for large and complex applications.
Technology stack is usually fixed.	Different services can use different technologies.
Maintenance becomes difficult as the application grows.	Easier maintenance due to smaller services.
Slower release cycles.	Faster development and release cycles.

8. Explain the advantages of Microservices in cloud-based applications.

Advantages of Microservices in Cloud-Based Applications

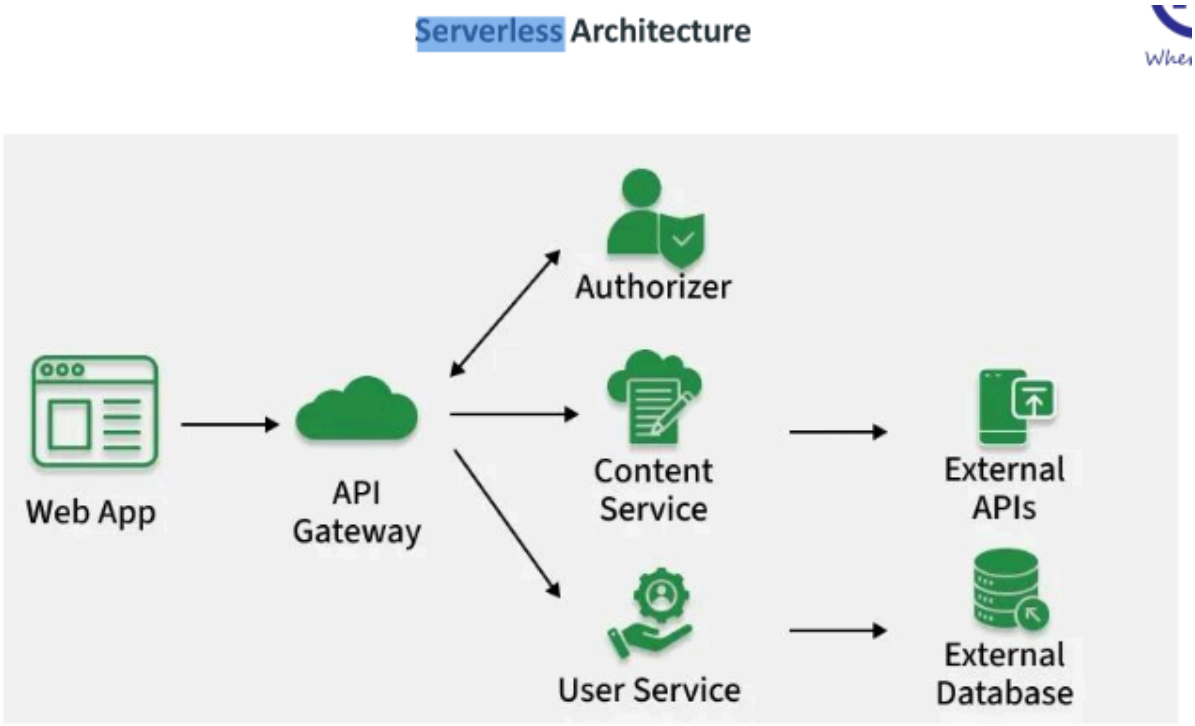
1. **Independent Deployment**
   - Each service can be deployed separately without affecting other services.
2. **Scalability**
   - Individual services can be scaled based on demand.
3. **Loose Coupling**
   - Services are independent and easier to develop, test, and maintain.
4. **Fault Isolation**
   - Failure of one service does not affect the entire application.
5. **Technology Flexibility**
   - Different services can use different programming languages and technologies.
6. **Faster Development**
   - Multiple teams can work on different services simultaneously.
7. **Ease of Maintenance**
   - Smaller codebases are easier to understand and modify.
8. **Improved Reliability**
   - Problems can be isolated and fixed without shutting down the whole system.
9. **Supports Continuous Delivery**
   - Enables frequent updates and faster releases.
10. **Better Resource Utilization**
    - Cloud resources can be allocated according to the needs of individual services.

From Serverless Architecture (Slides 72–82)

9. Explain the concept of Serverless Computing and discuss the steps involved in developing serverless applications.

Serverless Computing

Serverless Computing is a cloud computing model in which the cloud provider manages server provisioning, scaling, and maintenance. Developers focus only on writing and deploying code, while the cloud platform handles the infrastructure automatically.



Steps Involved in Developing Serverless Applications

1. **Understand the Serverless Model**
   - Analyze whether the application is suitable for serverless architecture.
   - Useful for event-driven and microservices-based applications.
2. **Choose the Right Provider**
   - Select a platform such as AWS Lambda, Azure Functions, Google Cloud Functions, or Oracle Cloud Functions.
3. **Design the Application**
   - Use event-driven and stateless design.
   - Plan functions and cloud services integration.
4. **Development Environment Setup**
   - Configure tools and local testing environments.

5. **Implement Functions**
    - Develop small, single-purpose functions.
    - Integrate databases, storage, and authentication services.
6. **Manage Dependencies**
    - Include only necessary libraries.
    - Reduce package size for better performance.
7. **Deployment and Continuous Integration**
    - Deploy functions to the cloud platform.
    - Automate deployment using CI/CD pipelines.

Advantages

1. No server management.
2. Automatic scaling.
3. Reduced operational costs.
4. Faster development and deployment.
5. Pay only for actual usage.

10. Discuss the challenges of implementing serverless applications.

Challenges of Implementing Serverless Applications

1. **Cold Start Delay**
    - The first request after a period of inactivity may experience a delay while the function starts.
2. **Limited Execution Environment**
    - Restrictions on memory, execution time, and supported languages.
3. **Debugging and Monitoring Difficulty**
    - Harder to trace and debug issues across multiple serverless functions.
4. **State Management Complexity**
    - Functions are stateless, so application state must be stored externally.
5. **Security and Compliance Challenges**
    - Additional effort is required to manage security, access control, and regulatory compliance.
6. **Vendor Lock-in**
    - Applications may become dependent on specific cloud provider services.
7. **Dependency Management**
    - Managing libraries and function dependencies can become complex in large applications.

Unit 4 Questions

From Static vs Dynamic Testing (UNIT_4-Static-vs-Dynamic-Testing.pptx)

1. Differentiate between Static Testing and Dynamic Testing with examples.

Difference Between Static Testing and Dynamic Testing

Static Testing	Dynamic Testing
Performed without executing the program.	Performed by executing the program.
Detects defects in documents, design, and source code.	Detects defects during software execution.
Performed in the early stages of development.	Performed after the software is developed.
Focuses on prevention of defects.	Focuses on finding defects in the running software.
Includes reviews, walkthroughs, and inspections.	Includes unit testing, integration testing, and system testing.
Does not require test data.	Requires test data and execution environment.
Less costly as defects are found early.	More costly if defects are found late.

Examples

Static Testing

- Requirement review
- Design review
- Code inspection

Dynamic Testing

1. Testing a login module
2. Unit testing
3. System testing

2. Discuss Model-Based Testing. Explain its advantages, disadvantages, and applications.

Model-Based Testing (MBT)

Model-Based Testing is a testing technique in which test cases are derived from models that describe the behavior, structure, or functionality of a system. The model represents how the system should behave, and test cases are generated from this model.

Advantages

1. Improves test coverage.
2. Detects defects early in the development process.
3. Reduces manual effort in test case generation.
4. Ensures systematic and consistent testing.
5. Supports automation of testing activities.

Disadvantages

1. Creating and maintaining models can be time-consuming.
2. Requires skilled personnel to develop models.
3. Initial setup cost is high.
4. Complex systems may require complex models.

Applications

1. Testing embedded systems.
2. Testing real-time systems.
3. Testing web applications.
4. Testing safety-critical systems.
5. Automated generation of test cases from UML models and state diagrams.

3. Explain the process of AI-driven software testing and discuss its benefits.

AI-Driven Software Testing

AI-driven software testing uses Artificial Intelligence and Machine Learning techniques to automate and improve software testing activities such as test case generation, test execution, defect prediction, and test maintenance.

Process of AI-Driven Software Testing

1. **Data Collection**
    - Gather requirements, test cases, source code, and defect data.
2. **Model Training**
    - Train AI/ML models using historical testing and defect data.
3. **Test Case Generation**
    - Automatically generate test cases based on requirements and system behavior.
4. **Test Execution**
    - Execute tests automatically and collect results.
5. **Defect Detection**
    - Identify failures, anomalies, and potential defects.
6. **Analysis and Feedback**
    - Analyze test results and improve future testing activities.

Benefits

1. Reduces manual testing effort.
2. Generates test cases automatically.
3. Improves test coverage.
4. Detects defects earlier.
5. Speeds up testing and software delivery.
6. Supports continuous testing in DevOps environments.
7. Improves accuracy and efficiency of testing.
8. Helps predict high-risk areas in software.

4. Explain AI techniques used in test case generation and defect prediction.

AI Techniques Used in Test Case Generation

1. **Machine Learning (ML)**
    - Learns from historical test cases and generates new test cases automatically.
2. **Natural Language Processing (NLP)**
    - Converts requirements written in natural language into test cases.
3. **Genetic Algorithms**
    - Generate optimized test cases by selecting the most effective test inputs.
4. **Neural Networks**
    - Learn system behavior and generate suitable test scenarios.

AI Techniques Used in Defect Prediction

1. **Machine Learning Classification**
    - Classifies software modules as defect-prone or non-defect-prone using historical data.
2. **Decision Trees**
    - Predict defects based on software metrics and previous defect patterns.
3. **Neural Networks**
    - Identify complex relationships between software attributes and defects.
4. **Random Forest**
    - Uses multiple decision trees to improve prediction accuracy.
5. **Deep Learning**
    - Analyzes large volumes of software data to identify potential defects.

Benefits

1. Improves testing efficiency.
2. Reduces manual effort.
3. Increases test coverage.
4. Detects high-risk modules early.
5. Enhances software quality and reliability.

5. Describe the role of Artificial Intelligence and Machine Learning in modern software testing.

Role of Artificial Intelligence and Machine Learning in Modern Software Testing

Artificial Intelligence (AI) and Machine Learning (ML) help automate and improve software testing by making testing faster, smarter, and more efficient.

Roles of AI and ML

1. **Automated Test Case Generation**
    - Generate test cases automatically from requirements, code, or user behavior.
2. **Defect Prediction**
    - Predict defect-prone modules using historical project data.
3. **Test Optimization**
    - Identify the most important test cases and reduce redundant tests.
4. **Automated Test Execution**
    - Execute tests automatically and analyze results.
5. **Self-Healing Test Scripts**
    - Automatically adapt test scripts when UI or application changes occur.

6. **Regression Testing**
    - Select and prioritize regression test cases efficiently.
7. **Anomaly Detection**
    - Detect unusual system behavior and potential failures.
8. **Bug Classification and Analysis**
    - Categorize defects and identify root causes quickly.
9. **Continuous Testing**
    - Support CI/CD pipelines through automated and intelligent testing.

Benefits

1. Reduces manual effort.
2. Improves test coverage.
3. Speeds up testing and release cycles.
4. Increases accuracy of defect detection.
5. Enhances software quality and reliability.
6. Supports faster decision-making in testing.

From Software Testing Fundamentals (Unit4_SOFTWARE_TESTING_FUNDAMENTALS.pptx)

6. Explain performance testing and discuss the various performance metrics used in software evaluation.

Performance Testing

Performance Testing is a type of software testing used to evaluate the speed, responsiveness, stability, scalability, and reliability of a system under a given workload.

Performance Metrics

1. **Response Time**
    - Time taken by the system to respond to a user request.
2. **Throughput**
    - Number of transactions or requests processed per unit time.
3. **Latency**
    - Delay between sending a request and receiving a response.
4. **Scalability**
    - Ability of the system to handle increasing workloads.
5. **Resource Utilization**
    - Usage of CPU, memory, disk, and network resources.
6. **Concurrency**
    - Number of users or transactions the system can handle simultaneously.

7. **Error Rate**
    - Percentage of failed requests or transactions.
8. **Availability**
    - Percentage of time the system remains operational and accessible.
9. **Reliability**
    - Ability of the system to perform consistently without failures.
10. **Load Handling Capacity**
    - Maximum workload that the system can support while maintaining acceptable performance.

7. Compare Volume Testing, Load Testing, and Stress Testing with examples.

Comparison of Volume Testing, Load Testing, and Stress Testing

Volume Testing	Load Testing	Stress Testing
Tests the system with a large volume of data.	Tests the system under expected user load.	Tests the system beyond its normal operating limits.
Focuses on data handling capability.	Focuses on system performance under workload.	Focuses on system stability and failure behavior.
Checks database size and data processing efficiency.	Checks response time and throughput.	Checks how the system behaves during overload.
Determines whether the system can manage large amounts of data.	Determines whether the system meets performance requirements.	Determines the breaking point of the system.
Normal user load may be present.	Expected number of users are simulated.	Excessive users or requests are simulated.

Examples

Volume Testing

- Testing a banking system with millions of customer records.

Load Testing

- Testing an e-commerce website with 5,000 concurrent users during a sale.

Stress Testing

- Testing the same website with 50,000 concurrent users to observe system failure and recovery behavior.

8. Explain scalability testing and discuss the key metrics used to evaluate system scalability.

Scalability Testing

Scalability Testing is a type of performance testing used to determine how well a system can handle increasing workloads, users, or data volumes without degrading performance.

Key Metrics Used to Evaluate Scalability

1. **Response Time**
    - Time taken by the system to respond to requests as workload increases.
2. **Throughput**
    - Number of transactions or requests processed per unit time.
3. **Concurrent Users**
    - Maximum number of users the system can support simultaneously.
4. **CPU Utilization**
    - Percentage of CPU resources used under increasing load.
5. **Memory Utilization**
    - Amount of memory consumed as the workload grows.
6. **Network Utilization**
    - Usage of network bandwidth during system operation.
7. **Database Performance**
    - Ability of the database to handle increasing data and transactions.
8. **Error Rate**
    - Number or percentage of failed requests under higher loads.
9. **Resource Efficiency**
    - Ability of the system to use resources effectively while scaling.

Benefits

1. Identifies system bottlenecks.
2. Ensures the system can support future growth.
3. Improves reliability and performance.
4. Helps in capacity planning.

9. Describe different types of scalability in distributed systems with suitable examples.

Types of Scalability in Distributed Systems

1. Vertical Scalability (Scale-Up)

Vertical scalability increases the capacity of a single server by adding more resources such as CPU, RAM, or storage.

Example:

- Upgrading a database server from 16 GB RAM to 64 GB RAM.

2. Horizontal Scalability (Scale-Out)

Horizontal scalability increases system capacity by adding more servers or nodes to the system.

Example:

- Adding additional web servers to handle increased traffic on an e-commerce website.

3. Diagonal Scalability

Diagonal scalability combines both vertical and horizontal scaling. A system is first upgraded with additional resources and then expanded by adding more servers when needed.

Example:

- Increasing the RAM of a server and later adding multiple servers to the cluster.

Benefits of Scalability

1. Supports increasing numbers of users.
2. Handles larger volumes of data.
3. Improves system performance and availability.
4. Enables future growth without major redesign.
5. Optimizes resource utilization.

10. Explain horizontal and vertical scalability with suitable examples.

Horizontal Scalability

Horizontal scalability (Scale-Out) increases system capacity by adding more servers or nodes to the system.

Advantages

1. Better fault tolerance.
2. Supports large numbers of users.
3. Easy to expand as demand increases.

Example

- Adding multiple web servers to an e-commerce website during a festive sale.

Vertical Scalability

Vertical scalability (Scale-Up) increases system capacity by adding more resources such as CPU, RAM, or storage to an existing server.

Advantages

1. Simple to implement.
2. No major changes to application architecture.
3. Suitable for small and medium-sized systems.

Example

- Upgrading a database server from 16 GB RAM to 64 GB RAM.

Difference Between Horizontal and Vertical Scalability

Horizontal Scalability	Vertical Scalability
Adds more servers or nodes.	Adds more resources to a single server.
Also called Scale-Out.	Also called Scale-Up.
Provides better fault tolerance.	Failure of the server affects the entire system.
Suitable for distributed systems.	Suitable for standalone systems.
Can handle very large workloads.	Limited by maximum hardware capacity.

Example: Adding more web servers.	Example: Increasing RAM or CPU of a server.

11. Discuss the impact of scalability testing on distributed systems.

Impact of Scalability Testing on Distributed Systems

Scalability Testing evaluates how effectively a distributed system can handle increasing workloads, users, or resources while maintaining acceptable performance.

Impacts

1. **Identifies System Bottlenecks**
    - Helps detect limitations in servers, databases, networks, or applications.
2. **Improves Performance**
    - Ensures the system maintains acceptable response time and throughput as demand increases.
3. **Supports Capacity Planning**
    - Helps determine the resources required for future growth.
4. **Enhances Reliability**
    - Ensures stable operation under varying workloads.
5. **Validates Horizontal and Vertical Scaling**
    - Verifies that the system can scale by adding resources or nodes.
6. **Optimizes Resource Utilization**
    - Helps use CPU, memory, storage, and network resources efficiently.
7. **Improves User Experience**
    - Maintains consistent performance for a large number of users.
8. **Reduces Risk of System Failure**
    - Identifies weaknesses before deployment in production environments.
9. **Supports Business Growth**
    - Ensures the distributed system can accommodate increasing users, transactions, and data volumes.

From Selenium, JUnit, TestNG & Cypress (UNIT4-Selenium_JUNIT_TestNG_Cypress.pptx)

12. Discuss the components of Selenium and explain how Selenium supports automated testing.

Components of Selenium

1. **Selenium IDE**
    - Record-and-playback tool used for creating simple test scripts.
2. **Selenium WebDriver**
    - Automates browser actions by directly interacting with web browsers.
    - Supports multiple programming languages and browsers.
3. **Selenium Grid**
    - Executes tests on multiple machines, browsers, and operating systems simultaneously.

How Selenium Supports Automated Testing

1. Automates web application testing.
2. Supports multiple browsers such as Chrome, Firefox, Edge, and Safari.
3. Supports multiple programming languages such as Java, Python, C#, and JavaScript.
4. Enables execution of repetitive test cases automatically.
5. Supports parallel test execution through Selenium Grid.
6. Integrates with testing frameworks such as TestNG and JUnit.
7. Supports Continuous Integration and Continuous Deployment (CI/CD) environments.
8. Reduces manual testing effort and testing time.
9. Improves test accuracy and repeatability.

13. Explain Selenium IDE, Selenium RC, Selenium Grid, and Selenium WebDriver.

Selenium IDE

- Selenium IDE (Integrated Development Environment) is a record-and-playback tool.
- It allows users to create test cases without programming.
- Suitable for simple web application testing.

Selenium RC (Remote Control)

- Selenium RC was an earlier Selenium tool used to automate web browsers.
- It allowed test scripts to be written in different programming languages.
- Selenium RC is now deprecated and replaced by Selenium WebDriver.

Selenium Grid

- Selenium Grid allows execution of test cases on multiple machines, browsers, and operating systems simultaneously.

- Supports parallel testing.
- Reduces test execution time.

Selenium WebDriver

- Selenium WebDriver is a browser automation tool that directly communicates with web browsers.
- Supports multiple browsers such as Chrome, Firefox, Edge, and Safari.
- Supports programming languages such as Java, Python, C#, and JavaScript.
- Provides faster and more reliable test execution than Selenium RC.

Difference Between Selenium Components

Component	Purpose
Selenium IDE	Record and playback of test cases
Selenium RC	Earlier browser automation tool (deprecated)
Selenium Grid	Parallel and distributed test execution
Selenium WebDriver	Browser automation through direct browser control

14. Explain Selenium WebDriver architecture.

Selenium WebDriver Architecture

Selenium WebDriver architecture consists of components that enable communication between test scripts and web browsers.

Architecture Components

1. **Test Script**
    - Written using a programming language such as Java, Python, C#, or JavaScript.
2. **WebDriver API**
    - Receives commands from the test script and forwards them to the browser driver.
3. **Browser Driver**
    - Acts as an intermediary between WebDriver and the browser.
    - Examples: ChromeDriver, GeckoDriver, EdgeDriver.
4. **Web Browser**
    - Executes the commands and performs actions such as clicking buttons, entering text, and navigating pages.

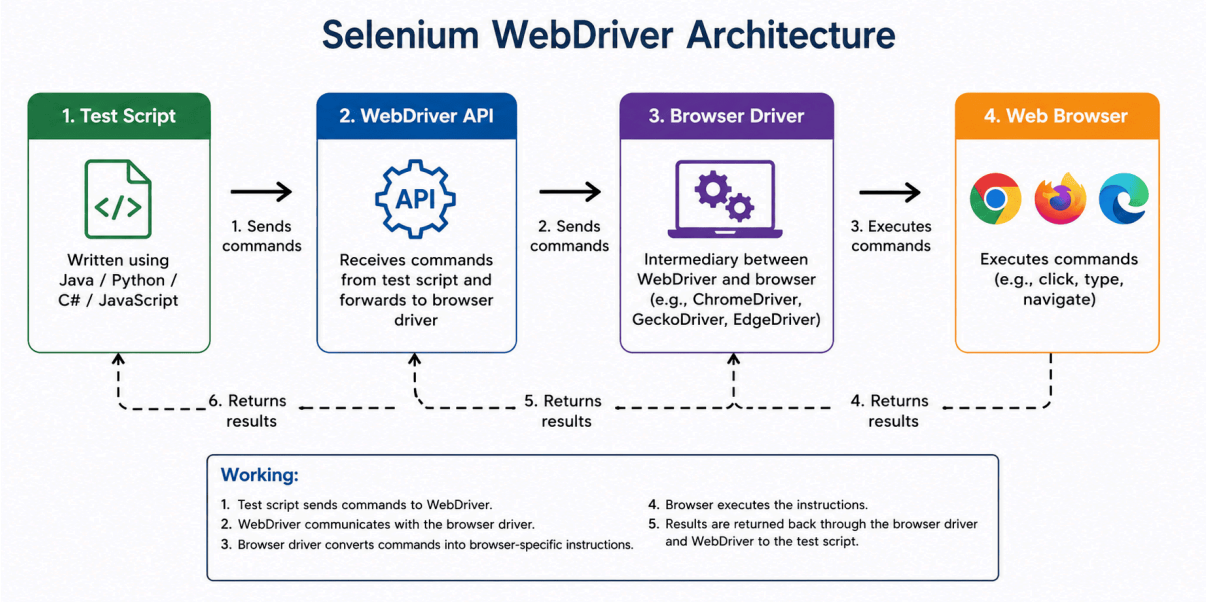
Working

1. Test script sends commands to WebDriver.
2. WebDriver communicates with the browser driver.
3. Browser driver converts commands into browser-specific instructions.
4. Browser executes the instructions.
5. Results are returned back through the browser driver and WebDriver to the test script.

Advantages

1. Direct communication with browsers.
2. Faster execution than Selenium RC.
3. Supports multiple browsers.
4. Supports multiple programming languages.
5. Enables reliable and efficient automated testing.

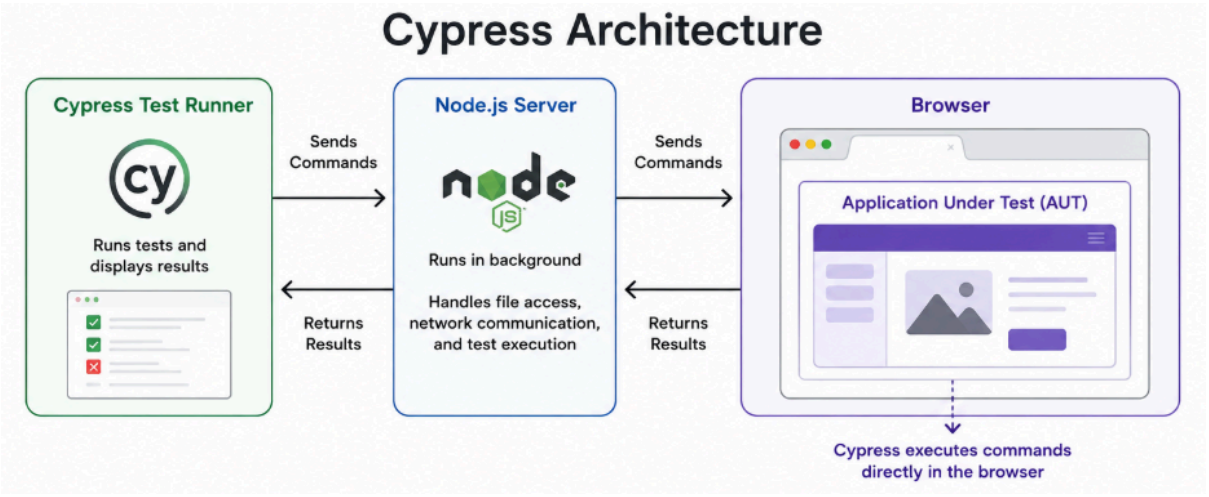
Diagram



15. Discuss Cypress architecture and its applications.

Cypress Architecture

Cypress is a modern end-to-end testing framework designed for web applications. Unlike Selenium, Cypress runs directly inside the browser and operates in the same execution loop as the application being tested.



Architecture Components

1. **Cypress Test Runner**
    - Executes test cases and displays results.
2. **Node.js Server**
    - Runs in the background.
    - Handles tasks such as file access, network communication, and test execution.
3. **Browser**
    - Cypress executes commands directly within the browser.
4. **Application Under Test (AUT)**
    - The web application being tested.

Working

1. Test scripts are written using JavaScript.
2. Cypress Test Runner executes the scripts.
3. Commands are sent to the browser.
4. Cypress interacts directly with the application.
5. Results are displayed in real time.

Applications of Cypress

1. End-to-end testing of web applications.
2. Integration testing.
3. UI testing.
4. API testing.
5. Regression testing.
6. Continuous testing in CI/CD pipelines.

Advantages

1. Fast execution.

2. Real-time test execution and debugging.
3. Automatic waiting for elements.
4. Easy setup and configuration.
5. Provides screenshots and videos for failed tests.

16. Explain JUnit annotations with examples.

JUnit Annotations

JUnit annotations are special markers used to control the execution of test methods and test classes.

Annotation	Purpose
@Test	Marks a method as a test method.
@BeforeEach	Executes before each test method.
@AfterEach	Executes after each test method.
@BeforeAll	Executes once before all test methods.
@AfterAll	Executes once after all test methods.
@Disabled	Skips the execution of a test method.

Example

```
import org.junit.jupiter.api.*;

public class CalculatorTest {

  @BeforeAll
  static void setup() {
    System.out.println("Before all tests");
  }

  @BeforeEach
  void init() {
    System.out.println("Before each test");
  }

  @Test
  void testAddition() {
    Assertions.assertEquals(5, 2 + 3);
  }
```

```
    @AfterEach
    void cleanup() {
        System.out.println("After each test");
    }

    @AfterAll
    static void finish() {
        System.out.println("After all tests");
    }
}
```

Output Order

1. @BeforeAll
2. @BeforeEach
3. @Test
4. @AfterEach
5. Repeat steps 2–4 for other test methods
6. @AfterAll

17. Describe various JUnit assertions used in unit testing.

JUnit Assertions

JUnit assertions are methods used to verify whether the actual result matches the expected result during unit testing.

Assertion	Purpose
assertEquals()	Checks whether expected and actual values are equal.
assertNotEquals()	Checks whether two values are not equal.
assertTrue()	Checks whether a condition is true.
assertFalse()	Checks whether a condition is false.
assertNull()	Checks whether an object is null.
assertNotNull()	Checks whether an object is not null.
assertSame()	Checks whether two references point to the same object.
assertNotSame()	Checks whether two references point to different objects.
assertArrayEquals()	Checks whether two arrays are equal.

Examples

```
assertEquals(10, 5 + 5);

assertNotEquals(10, 5 + 3);

assertTrue(10 > 5);

assertFalse(5 > 10);

assertNull(null);

assertNotNull("JUnit");

assertArrayEquals(
  new int[]{1,2,3},
  new int[]{1,2,3}
);
```

Benefits

1. Verifies expected results automatically.
2. Helps identify defects quickly.
3. Improves reliability of unit tests.
4. Simplifies test validation.

Unit 5 Questions

From Performance Testing (Pages 2–19)

1. Explain performance testing and discuss the various performance metrics used in software evaluation.

Performance Testing

Performance Testing is a type of software testing used to evaluate the speed, responsiveness, stability, scalability, and reliability of a system under a given workload.

Performance Metrics

1. **Response Time**
    - Time taken by the system to respond to a user request.
2. **Throughput**

    - Number of transactions or requests processed per unit time.
3. **Latency**
    - Delay between sending a request and receiving a response.
4. **Scalability**
    - Ability of the system to handle increasing workloads.
5. **Resource Utilization**
    - Usage of CPU, memory, disk, and network resources.
6. **Concurrency**
    - Number of users or transactions the system can handle simultaneously.
7. **Error Rate**
    - Percentage of failed requests or transactions.
8. **Availability**
    - Percentage of time the system remains operational and accessible.
9. **Reliability**
    - Ability of the system to perform consistently without failures.
10. **Load Handling Capacity**
    - Maximum workload that the system can support while maintaining acceptable performance.

2. Compare Volume Testing, Load Testing, and Stress Testing with examples.

Comparison of Volume Testing, Load Testing, and Stress Testing

Volume Testing	Load Testing	Stress Testing
Tests the system with a large volume of data.	Tests the system under expected user load.	Tests the system beyond its normal operating limits.
Focuses on data handling capability.	Focuses on system performance under workload.	Focuses on system stability and failure behavior.
Checks database size and data processing efficiency.	Checks response time, throughput, and resource usage.	Checks how the system behaves during overload conditions.
Determines whether the system can manage large amounts of data.	Determines whether the system meets performance requirements.	Determines the breaking point of the system.
Normal user load may be present.	Expected number of users are simulated.	Excessive users or requests are simulated.

Examples

Testing Type	Example
Volume Testing	Testing a banking system with millions of customer records.
Load Testing	Testing an e-commerce website with 5,000 concurrent users during a sale.
Stress Testing	Testing the same website with 50,000 concurrent users to observe failure and recovery behavior.

From Security Testing & OWASP (Pages 20–46)

3. Explain OWASP Top 10 web application security risks and methods to mitigate them.

OWASP Top 10 Web Application Security Risks

OWASP Top 10 is a list of the most critical web application security risks.

Risk	Description	Mitigation
Broken Access Control	Users can access unauthorized resources or functions.	Implement proper authorization and access control checks.
Cryptographic Failures	Sensitive data is exposed due to weak encryption.	Use strong encryption and secure key management.
Injection	Malicious code such as SQL injection is executed by the application.	Validate inputs and use parameterized queries.
Insecure Design	Security is not considered during application design.	Apply secure design principles and threat modeling.
Security Misconfiguration	Incorrect security settings expose vulnerabilities.	Use secure configurations and regular security reviews.
Vulnerable and Outdated Components	Use of outdated libraries and software.	Regularly update and patch components.

Identification and Authentication Failures	Weak authentication mechanisms allow unauthorized access.	Use strong passwords, MFA, and secure session management.
Software and Data Integrity Failures	Untrusted software updates or data modifications.	Verify software integrity and use secure update mechanisms.
Security Logging and Monitoring Failures	Security incidents are not detected or investigated.	Enable logging, monitoring, and alerting mechanisms.
Server-Side Request Forgery (SSRF)	Server makes unintended requests to internal or external resources.	Validate URLs, restrict outbound requests, and use network controls.

Benefits of Mitigation

1. Protects sensitive data.
2. Reduces security vulnerabilities.
3. Prevents unauthorized access.
4. Improves application reliability and trustworthiness.
5. Enhances overall web application security.

4. Discuss the importance of security testing in software development.

Importance of Security Testing in Software Development

Security Testing is the process of identifying vulnerabilities, threats, and risks in a software application and ensuring that data and resources are protected from unauthorized access.

Importance

1. **Protects Sensitive Data**
    - Prevents unauthorized access to confidential information.
2. **Identifies Security Vulnerabilities**
    - Detects weaknesses before the software is deployed.
3. **Prevents Cyber Attacks**
    - Reduces risks from attacks such as SQL injection, XSS, and malware.
4. **Ensures Data Integrity**
    - Protects data from unauthorized modification or deletion.
5. **Maintains Confidentiality**

    - Ensures information is accessible only to authorized users.
6. **Improves System Reliability**
    - Enhances the overall stability and trustworthiness of the application.
7. **Supports Compliance**
    - Helps meet security standards and regulatory requirements.
8. **Reduces Financial Loss**
    - Prevents losses caused by security breaches and data leaks.
9. **Enhances Customer Trust**
    - Increases user confidence in the software product.
10. **Ensures Business Continuity**
    - Minimizes disruptions caused by security incidents.

5. Discuss common web application vulnerabilities and security testing techniques used to identify and mitigate them.

Common Web Application Vulnerabilities

1. **SQL Injection**
    - Attackers insert malicious SQL queries through input fields.
2. **Cross-Site Scripting (XSS)**
    - Malicious scripts are injected into web pages and executed in users' browsers.
3. **Cross-Site Request Forgery (CSRF)**
    - Unauthorized actions are performed on behalf of authenticated users.
4. **Broken Authentication**
    - Weak authentication mechanisms allow unauthorized access.
5. **Broken Access Control**
    - Users gain access to resources beyond their permissions.
6. **Security Misconfiguration**
    - Improper security settings expose the application to attacks.
7. **Sensitive Data Exposure**
    - Confidential information is disclosed due to inadequate protection.
8. **Server-Side Request Forgery (SSRF)**
    - The server is tricked into making unintended requests.

Security Testing Techniques

1. **Vulnerability Scanning**
    - Automated tools identify known security weaknesses.
2. **Penetration Testing**
    - Simulated attacks are performed to find exploitable vulnerabilities.
3. **Security Auditing**
    - Review of source code, configurations, and security policies.
4. **Risk Assessment**
    - Identifies and prioritizes security risks.
5. **Static Application Security Testing (SAST)**

    - Analyzes source code without executing the application.
6. **Dynamic Application Security Testing (DAST)**
    - Tests the running application for vulnerabilities.

Mitigation Methods

1. Validate and sanitize user inputs.
2. Use parameterized queries to prevent SQL injection.
3. Implement strong authentication and authorization.
4. Encrypt sensitive data.
5. Apply secure configuration settings.
6. Regularly update and patch software components.
7. Enable logging and monitoring.
8. Conduct regular security testing and audits.

From Scalability Testing (Pages 47–58)

6. Explain scalability testing and discuss the key metrics used to evaluate system scalability.

Scalability Testing

Scalability Testing is a type of performance testing used to determine how well a system can handle increasing workloads, users, or data volumes without degrading performance.

Key Metrics Used to Evaluate System Scalability

1. **Response Time**
    - Time taken by the system to respond to user requests as load increases.
2. **Throughput**
    - Number of transactions or requests processed per unit time.
3. **Concurrent Users**
    - Maximum number of users the system can support simultaneously.
4. **CPU Utilization**
    - Percentage of CPU resources used under increasing workload.
5. **Memory Utilization**
    - Amount of memory consumed as the workload grows.
6. **Network Utilization**
    - Usage of network bandwidth during operation.
7. **Database Performance**
    - Ability of the database to handle increasing transactions and data.
8. **Error Rate**
    - Percentage of failed requests under heavy load.
9. **Resource Efficiency**
    - Effectiveness of resource usage while scaling.

Benefits

1. Identifies performance bottlenecks.
2. Supports capacity planning.
3. Ensures system reliability under growth.
4. Improves user experience.
5. Validates system scalability requirements.

7. Describe different types of scalability in distributed systems with suitable examples.

Types of Scalability in Distributed Systems

1. Vertical Scalability (Scale-Up)

Vertical scalability increases the capacity of a single server by adding more resources such as CPU, RAM, or storage.

Example

- Upgrading a database server from 16 GB RAM to 64 GB RAM.

2. Horizontal Scalability (Scale-Out)

Horizontal scalability increases system capacity by adding more servers or nodes to the system.

Example

- Adding additional web servers to handle increased traffic on an e-commerce website.

3. Diagonal Scalability

Diagonal scalability combines both vertical and horizontal scaling. The system is first upgraded with additional resources and later expanded by adding more servers when required.

Example

- Increasing the RAM of a server and then adding multiple servers to a cluster as user demand grows.

Benefits

1. Supports increasing numbers of users.
2. Handles larger volumes of data.

3. Improves performance and availability.
4. Enables future growth.
5. Optimizes resource utilization.

8. Explain horizontal and vertical scalability with suitable examples.

Horizontal Scalability

Horizontal scalability (Scale-Out) increases system capacity by adding more servers or nodes to the system.

Advantages

1. Better fault tolerance.
2. Supports large numbers of users.
3. Easy to expand as demand increases.

Example

- Adding multiple web servers to an e-commerce website during a festive sale.

Vertical Scalability

Vertical scalability (Scale-Up) increases system capacity by adding more resources such as CPU, RAM, or storage to an existing server.

Advantages

1. Simple to implement.
2. No major changes to application architecture.
3. Suitable for small and medium-sized systems.

Example

- Upgrading a database server from 16 GB RAM to 64 GB RAM.

Difference Between Horizontal and Vertical Scalability

Horizontal Scalability	Vertical Scalability
Adds more servers or nodes.	Adds more resources to a single server.
Also called Scale-Out.	Also called Scale-Up.
Provides better fault tolerance.	Failure of the server affects the entire system.

Suitable for distributed systems.	Suitable for standalone systems.
Can handle very large workloads.	Limited by maximum hardware capacity.
Example: Adding more web servers.	Example: Increasing RAM or CPU of a server.

9. Discuss the impact of scalability testing on distributed systems.

Impact of Scalability Testing on Distributed Systems

Scalability Testing evaluates how effectively a distributed system can handle increasing workloads, users, or resources while maintaining acceptable performance.

Impacts

1. **Identifies System Bottlenecks**
    - Helps detect limitations in servers, databases, networks, and applications.
2. **Improves Performance**
    - Ensures acceptable response time and throughput as workload increases.
3. **Supports Capacity Planning**
    - Helps estimate future hardware and software resource requirements.
4. **Enhances Reliability**
    - Ensures stable operation under varying workloads.
5. **Validates Scaling Mechanisms**
    - Confirms that horizontal and vertical scaling work effectively.
6. **Optimizes Resource Utilization**
    - Helps use CPU, memory, storage, and network resources efficiently.
7. **Improves User Experience**
    - Maintains consistent performance for a growing number of users.
8. **Reduces Risk of Failures**
    - Identifies weaknesses before deployment in production environments.
9. **Supports Business Growth**
    - Ensures the system can accommodate increasing users, transactions, and data volumes.